Vector Extension Support for the Kites RISC-V Simulator

Design Document

RISC-V "V" extension 1.0, the Zve* subsets, and the Zvfh, Zvbb/Zvkb and Zvbc extensions

Document	Design document
Version	1.0
Author	Raam M
Repository	`https://github.com/Raam-M/Kites`, branch `vector-extension`
Companions	Scope document (requirements, decisions D1–D14, findings F1–F19); validation report (results)

Purpose of this document

This document describes how the vector extension is built into Kites: the data structures, the execution model, how each instruction family is implemented, how vector instructions run on the single-cycle and the pipelined processors, the configuration options, the statistics and timing model, and the user interfaces. It records the design decisions and the known limitations. The *why* of the scope is in the scope document; the evidence that the design works is in the validation report.

Contents

1 Overview **3**

2 Instruction set table **4**

2.1 Generation 4

2.2 Run-time table 4

3 Assembler and disassembler **5**

4 Architectural state **5**

5 Execution model **6**

5.1 One instruction, one atomic step 6

5.2 Element loop, masking and policies 6

5.3 Register groups 6

6 Instruction families **6**

6.1 Configuration 6

6.2 Loads and stores 7

6.3 Integer and fixed point 7

6.4 Floating point 7

6.5 Half precision (Zvfh) 7

6.6 Unordered reductions 8

6.7 Bit manipulation and carry-less multiply 8

7 Configuration **8**

8 Pipelined execution **9**

8.1 Execute in write-back 9

8.2 Hazards 9

8.3 CSR instructions 10

8.4 Scalar defects fixed along the way 10

9 Statistics and timing model **10**

9.1 Statistics 10

9.2 Timing model 10

9.3 Undo and reset 11

10 User interfaces **11**

11 Design decisions 12

12 Known limitations 12

13 Source files 13

1 Overview

Kites now implements the complete RISC-V vector extension 1.0 [1]: every one of the 627 **V** instructions (375 base encodings and 252 segment variants), the standard vector pseudo-instructions, and the seven vector CSRs. On top of that it implements:

- the embedded subsets Zve64d, Zve64f, Zve64x, Zve32f and Zve32x as selectable profiles;
- Zvfh (half-precision floating point), Zvbb and its subset Zvkb (bit manipulation) and Zvbc (carry-less multiply), 20 further base encodings;
- VLEN from 128 to 1024 bits and a choice of agnostic-element policy;
- execution on the single-cycle processor and on all four 5-stage pipeline models;
- vector statistics and an analytic timing model with lanes and chaining;
- a vector register viewer, a settings page, vector-unit wires in the circuit view, vector commands in the CLI REPL, and vector instructions in custom pseudo-instructions.

Figure 1 shows the parts and how they connect. The design rests on three ideas:

1. **One ISA table.** A table generated from riscv-opcodes [2] describes every vector instruction: name, fixed bits, operand fields and defining extension. The assembler, disassembler, executor dispatch, editor support and tests all read it (D10).
2. **Atomic execution.** A vector instruction reads a snapshot of the state, checks every legality rule, computes its results into a staging area and only then commits them in one step, recording each change for undo (D5). An illegal instruction therefore changes nothing.
3. **Processor-independent vector unit.** The unit is a function, `rvv::execute(word, context)`, that sees the register file, the memory controller and the step record. The single-cycle processor calls it in its execute step; the pipelines call it in write-back (Section 8).

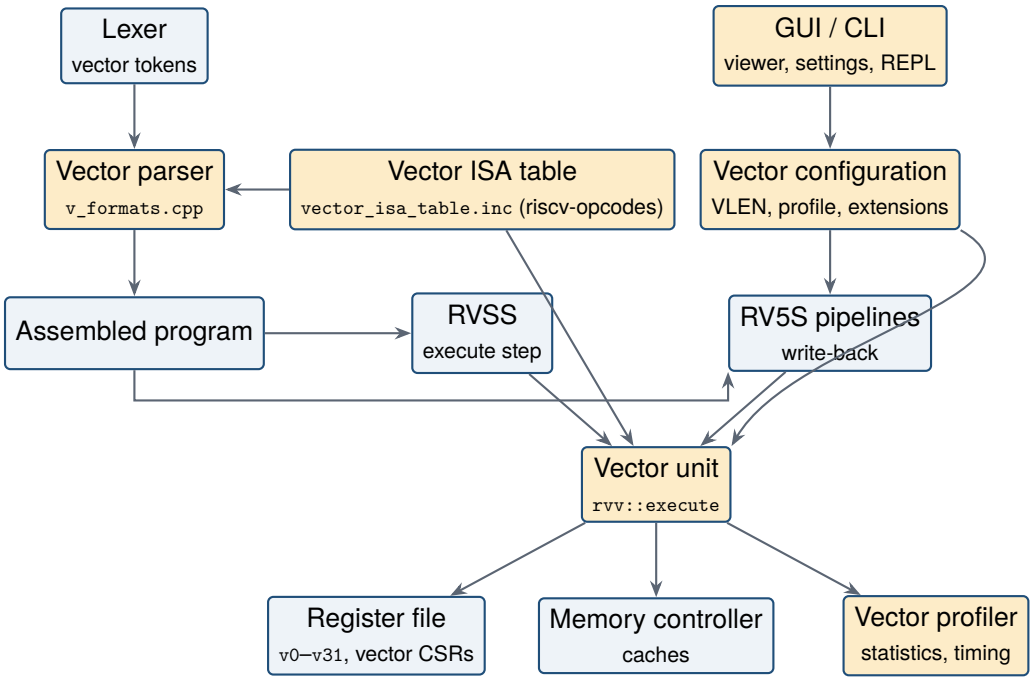


Figure 1: Architecture. Shaded yellow: new components; blue: existing components with hooks.

2 Instruction set table

2.1 Generation

`tools/vector/gen_vector_isa.py` reads the riscv-opcodes files `rv_v`, `rv_zvbb` and `rv_zvbc` and writes `src/processor/vector/vector_isa_table.inc`, one line per base encoding:

```
KITES_RVV_INSN("vadd.vv", 0x00000057u, 0xfc00707fu, kVm | kVs2 | kVs1 | kVd, kExtV)
KITES_RVV_INSN("vror.vi", 0x50003057u, 0xf800707fu, kZimm6 | kVm | kVs2 | kVd, kExtZvkb)
```

`match` holds the fixed bits, `mask` marks them, the fourth argument lists the variable fields and the last one names the defining extension. The generator checks that the fixed and variable fields cover all 32 bits exactly and that no instruction word matches two entries. Zvkb is the subset of Zvbb that riscv-opcodes' `rv_zvkn` imports (`vandn`, `vbrev8`, `vrev8`, `vrol`, `vror`).

2.2 Run-time table

`vector_isa.cpp` expands the 395 base entries into 647 `InsnInfo` records: each unit-stride, strided and indexed load or store with an `nf` field gets seven segment variants (`vlseg2e8.v` … `vlseg8e8.v`). Each record holds the mnemonic, its base and suffix, `match`/`mask`, the field set, the major opcode and `funct3`, the segment count, the extension, and the ordered list of *assembly operands* derived from the fields (Table 1). Multiply-add instructions list the multiplier before `vs2`, as the assembly syntax requires.

Table 1: Operand kinds derived from the fields.

Kind	Fields	Assembly form
Vector register	`vd`, `vs3`, `vs1`, `vs2`	`v0`–`v31`
Integer register	`rd`, `rs1`, `rs2`	`x0`–`x31` or ABI names
FP register	`rd`/`rs1` with OPFVV/OPFVF	`f0`–`f31` or ABI names
Memory	`rs1` of a load/store	`(rs1)` or `0(rs1)`
Immediate	`simm5`, `zimm5`, `zimm6`	$-16..15$, $0..31$, $0..63$
vtype immediate	`zimm10`, `zimm11`	`e32, m2, ta, ma`
`v0`	fixed `vm`=0	carry-in or merge selector
Optional mask	`vm`	optional `, v0.t`

`zimm6` (`vror.vi`) is split over bit 26 and bits 19..15; `getField` and `setField` handle it as one 6-bit value. `decode(word)` finds the entry whose `mask`/`match` fits the word. `isVectorInstruction(word)` recognises the vector encoding space: all of OP-V, and LOAD-FP/STORE-FP with widths 000, 101, 110 and 111. Widths 010 and 011 remain `flw`/`fld`/`fsw`/`fsd` (F6).

3 Assembler and disassembler

Lexing. The lexer produces `VEC_REGISTER` for `v0`–`v31`, `VEC_MASK` for `v0.t`, and `VTYPE_FIELD` for `e8`...`e64`, `mf8`...`m8`, `ta`/`tu`, `ma`/`mu` only on lines whose opcode is `vsetvli` or `vsetivli`. Elsewhere those words remain label references, so a label called `ta` still works (AS-04).

Parsing. `Parser::parse_vector` handles every vector mnemonic generically. It splits the operand tokens into comma-separated groups and matches them against the instruction's operand list (`v_formats.cpp`, `encodeVector`). Each group must have the right kind; an immediate is checked against its range; the optional mask may be present only where `vm` is a field. The word is built from `match` with `setField` and attached to the instruction unit, so the code generator emits it unchanged. Errors report the line, the column and the expected syntax from `expectedSyntax`, for example "`vadd.vv vd, vs2, vs1[, v0.t]`".

Pseudo-instructions. `parse_vector_pseudo` expands the standard vector pseudo-instructions (`vneg.v`, `vnot.v`, `vwcvt.x.x.v`, `vncvt.x.x.w`, `vfneg.v`, `vfabs.v`, `vmmv.m`, `vmnot.m`, `vmclr.m`, `vmset.m`, the reversed compares, `vmsge(u).vx` with its masked and temporary-register forms, and `vl<n>r.v`). `parse_csr_pseudo` adds `csrr`, `csrw`, `csrs`, `csrc` and their immediate forms, with vector CSR names.

Disassembly. `rvv::disassemble` prints the mnemonic and the operands in table order. The disassembly view, the register dump and the error messages use it. Assembling its output gives the same word (AS-02).

4 Architectural state

`RegisterFile` holds the vector registers as one byte array of $32 \times$ VLENB bytes. Element i of a register occupies the bytes $i \cdot \mathrm{SEW}/8$ upward, little-endian, so element 0 is in the lowest bytes. A register group of LMUL registers is contiguous in this array, which makes the group addressing of Section 5 a simple offset calculation. `ReadVr`/`WriteVr` copy one register; `GetVrBytes` exposes the whole file to the viewer.

Table 2: Vector CSRs.

CSR	Address	Access	Implementation
`vstart`	0x008	read/write	reset to 0 after every vector instruction
`vxsat`	0x009	read/write	bit 0 only
`vxrm`	0x00A	read/write	bits 1:0 only
`vcsr`	0x00F	read/write	composite: reads `(vxrm << 1) | vxsat`, writes both
`vl`	0xC20	read-only	written only by `vset*` and fault-only-first loads
`vtype`	0xC21	read-only	`vill` in bit 63
`vlenb`	0xC22	read-only	VLEN/8, set when the register file is reset

Writing a read-only CSR (address bits 11:10 = 11) is an illegal instruction on every processor. Reset sets `vtype.vill`=1 and `vl`=0 and clears the vector registers (D4). VLEN is read from the configuration on reset, so a new VLEN applies at the next reset or assemble.

5 Execution model

5.1 One instruction, one atomic step

`rvv::execute(word, context)` runs one instruction. The context holds references to the register file, the memory controller and the current step record (`StepDelta`), and optionally the profiler and the processor cycle. Execution has three phases:

1. **Set-up.** An `ExecState` decodes the fields, reads `vtype`, `vl`, `vstart` and VLEN, and takes a snapshot of the vector registers (`source`) and a working copy (`destination`). The instruction's extension must be enabled (Section 7).
2. **Checks and computation.** The family handler checks every legality rule first: `vill`, register-group alignment (`checkGroup`), EMUL within $[1/8, 8]$, the overlap rules (`checkOverlap`, `forbidOverlap`, `checkMaskOverlap`), `vstart`=0 where required, and the supported SEW. Then it loops over the elements. Reads come from the snapshot and writes go to the working copy, so an instruction whose source and destination overlap reads the old values. Scalar results, CSR updates, `fflags` and stores are staged.
3. **Commit.** `commit` writes the changed vector registers, the staged scalar register, the CSRs and the memory, and records every old and new value in the step record. It then sets `vstart` to 0.

A rule violation throws `IllegalInstruction`, and an access outside memory throws `AccessFault`, before phase 3, so nothing has changed (D5). The processor leaves the PC on the instruction, and Kites shows the message with the source line.

5.2 Element loop, masking and policies

For every body element i with `vstart` $\leq i <$ `vl`, the handler asks `active(i)`: true when the instruction is unmasked or bit i of `v0` is set. Active elements get the result. Inactive elements and tail elements ($i \geq$ `vl`) go through `inactive()` and `tail()`, which apply the policy: undisturbed leaves the element alone; with the all-ones fill selected (Section 7) and `vma`/`vta` set, the element becomes all ones. Mask-producing instructions treat their tail as agnostic, as the specification requires. Elements below `vstart` are never touched.

5.3 Register groups

`group(reg, eew)` returns the register, the element width and EMUL $=$ (EEW/SEW) $\cdot$ LMUL. Element i of a group lives at byte offset `reg` $\cdot$ VLENB $+ i \cdot$ EEW$/8$ of the register array, so groups need no special indexing. Widening, narrowing, extension, index and mask operands all use the same accessor with their own EEW.

6 Instruction families

6.1 Configuration

`vsetvli`, `vsetivli` and `vsetvl` compute VLMAX $=$ LMUL $\cdot$ VLEN/SEW and set `vl` $=$ min(AVL, VLMAX) (D2). `rs1=x0` with `rd`$\neq$`x0` requests VLMAX; `rs1=rd=x0` keeps `vl` and sets `vill` if VLMAX would change. A vtype with reserved bits, a reserved `vlmul`, SEW above ELEN, or SEW $>$ LMUL $\cdot$ ELEN sets `vill`=1 and `vl`=0 (D4). ELEN depends on the profile, so

under Zve32* SEW=64 and LMUL=1/8 are reserved.

6.2 Loads and stores

`vector_exec_memory.cpp` handles unit-stride, strided, indexed (ordered and unordered), segment, fault-only-first, mask and whole-register accesses. Each element is a separate access through the `MemoryController` in increasing element (and field) order, so caches see the traffic and the ordered and unordered indexed forms behave the same (D8). Misaligned element addresses are allowed. Loads write the working copy; stores are staged and committed with the rest of the instruction. A fault-only-first load that faults on element $i > 0$ trims `vl` to i; on element 0 it is an access fault. An EEW above ELEN is illegal.

6.3 Integer and fixed point

`vector_exec_integer.cpp` dispatches on the base mnemonic through tables of element operations: single-width arithmetic, multiply-add, widening and narrowing, extension, carry and borrow, compares, merges and moves, reductions, mask logical operations, `vcpop.m`, `vfirst.m`, `vmsbf/vmsif/vmsof`, `viota`, `vid`, slides, gathers, compress and whole-register moves. Division by zero and signed overflow give the RISC-V results. The high halves of 64-bit products use portable 64×64-bit helpers, not `__int128` (F9). Fixed-point instructions round with `vxrm` (rnu, rne, rdn, rod) and set `vxsat` on saturation.

6.4 Floating point

`vector_exec_float.cpp` does not use the scalar FP unit, which deviates from the specification (F16). For SEW = 32 and 64 it uses host IEEE 754 arithmetic: the rounding mode comes from `frm` (`HostRounding`), the host exception flags are cleared before and read after each element, and they are accrued into `fflags` in the RISC-V bit order (D9). On top of that it applies the RISC-V rules explicitly: canonical NaN results; minimumNumber and maximumNumber for `vfmin/vfmax`; quiet and signalling compares; saturating float-to-integer conversion; round-to-odd for `vfncvt.rod.f.f.w`; and the 7-bit estimates `vfrec7/vfrsqrt7` from the specification's tables, bit-exact including their special cases. The code is templated on a format descriptor, so each operation is written once for all widths.

6.5 Half precision (Zvfh)

There is no portable half-precision host type, so binary16 is handled in software. A half operation converts its operands exactly to double, computes the operation in double under the same rounding mode, and rounds the double result to half with `roundToHalf`. That function implements all four host rounding modes and raises inexact, overflow and underflow with tininess detected *after rounding*, as RISC-V requires.

The double rounding does not change the result. Sums, differences, products and integer conversions of halves are exact in double. For division and square root, rounding first to a format with $p' \geq 2p+2$ significand bits and then to p bits gives the correctly rounded result [5]; here $p = 11$ and $p' = 53$. For fused multiply-add, the product of two halves is exact in double; the sum is rounded once to double and then to half. A difference could only arise if the two addends were more than 53 bit positions apart and the larger one lay exactly halfway between two halves. The larger addend is then either a half, which cannot lie halfway, or a product of at least 2^{29}, which overflows half anyway. Under the directed modes, both roundings go in the same direction. A signalling NaN

operand raises invalid explicitly, because the software conversion does not. Widening operations convert halves exactly to single and then use the single-precision path; narrowing conversions round single to half with `roundToHalf`.

6.6 Unordered reductions

`vfredusum` and `vfwredusum` add in element order, which is one of the orders the specification allows. When no element is active, the specification also allows adding an additive identity to `vs1[0]`. Kites makes the same choice as Spike: a NaN scalar becomes the canonical NaN, and a signalling NaN raises invalid.

6.7 Bit manipulation and carry-less multiply

Zvkb adds `vandn`, `vrol`, `vror` (with the 6-bit `vror.vi` immediate), `vbrev8` and `vrev8`. Zvbb adds `vbrev`, `vclz`, `vctz`, `vcpop.v` and the widening shift `vwsll`, which zero-extends `vs2` and shifts by the low $\log_2(2 \cdot \mathrm{SEW})$ bits of the operand. Zvbc adds `vclmul` and `vclmulh`, defined only for SEW=64. The binary operations reuse the single-width loop; the unary ones have their own small handler.

7 Configuration

`rvv::VectorConfig` (`vector_config.h`) holds the run-time options. The CLI flags, the GUI settings page and the REPL command `vconfig` all write the same structure.

Table 3: Configuration options.

Option	Values	Effect
VLEN	128, 256, 512, 1024	register width; applied at the next reset
Agnostic fill	undisturbed, ones	value of agnostic tail and inactive elements (D3)
Profile	V, Zve64d/f/x, Zve32f/x	ELEN, FP widths, 64-bit high multiplies (Table 4)
Extensions	Zvfh, Zvbb, Zvkb, Zvbc	enable the instructions; Zvbb implies Zvkb
Timing lanes	1, 2, ..., 32	timing model (Section 9)
Chaining	on, off	timing model

Table 4: Profiles. A "–" means the restriction does not apply because SEW=64 is not available.

Profile	ELEN	FP element widths	`vmulh*`, `vsmul` at SEW=64
V	64	32, 64 (+16 with Zvfh)	yes
Zve64d	64	32, 64 (+16 with Zvfh)	no
Zve64f	64	32 (+16 with Zvfh)	no
Zve64x	64	none	no
Zve32f	32	32 (+16 with Zvfh)	–
Zve32x	32	none	–

`validate` rejects combinations the specification does not allow: Zvfh without an F profile, and Zvbc with ELEN=32. An instruction of a disabled extension, an FP instruction at an unsupported width, or an EEW above ELEN is an illegal instruction. The assembler accepts every instruction regardless of the configuration, as hardware assemblers do with the right flags; legality is decided when the instruction executes.

8 Pipelined execution

8.1 Execute in write-back

On the four 5-stage models (no hazard detection or forwarding; hazard detection without forwarding; forwarding without hazard detection; both), vector instructions and Zicsr CSR instructions execute atomically in **WB**. Decode treats them like a bubble with no control signals; they pass through EX and MEM without effect; `pipeline_writeback` calls `rvv::execute` (or the CSR routine) for the instruction in MEM/WB before anything else in the cycle. This keeps the vector unit identical for all processors and gives four properties without extra logic:

Scalar operands are current.
Every older instruction has already written back, so the vector unit reads its scalar operands from the register file with no forwarding.

Memory order is program order.
A Kites step runs WB before MEM. An older scalar store has finished its MEM stage in an earlier cycle. A younger scalar load or store runs its MEM stage in the same or a later cycle, after the vector access.

Exceptions are precise.
Write-back runs first in a step. An illegal vector instruction stops the step before any younger instruction has written anything. The PC is set to the instruction for the error report, and stepping again reports the same error.

Wrong-path instructions never execute.
Branches resolve in MEM and jumps in EX. Both flush the younger stages before those instructions reach WB.

8.2 Hazards

A vector or CSR instruction never waits for an older instruction. A younger scalar instruction must wait in two cases, checked in IF/ID after the existing hazard unit (`writeback_dependency_stall`):

1. it reads a register that an older vector or CSR instruction still in EX/MEM or MEM/WB writes (`rd` of `vset*`, `vmv.x.s`, `vcpop.m`, `vfirst.m`, `csrr*`; the FP register of `vfmv.f.s`);
2. it is a scalar FP operation, which reads `frm` and writes `fflags` in EX, while any vector or CSR instruction is in flight.

The stall is re-evaluated every cycle. A value produced in WB cannot be forwarded, so a dependent instruction waits two cycles on both hazard-detecting models, as for a load without forwarding. The models without hazard detection do not stall: the program must separate such a producer and its consumer by two instructions, the same rule as for a load on those models.

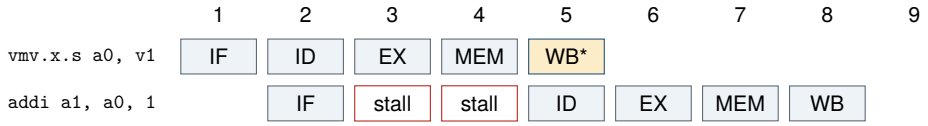


Figure 2: A scalar instruction that uses a vector result. WB* is where the vector instruction executes; WB runs first in a cycle, so ID in cycle 5 reads the new `a0`.

8.3 CSR instructions

CSR instructions used to be no-ops on the pipelines. They now execute in WB through the same path as vector instructions, with the same read-only check as the single-cycle processor, so programs can set `vxrm` or read `vl` and `vlenb`.

8.4 Scalar defects fixed along the way

Running vector programs on the pipelines exposed three existing defects, fixed in separate commits with regression tests: AUIPC computed a register sum instead of PC+imm (so `la` gave wrong addresses on every pipeline) and LUI did not sign-extend; the forwarding models did not forward the integer source of F/D instructions (load/store base, `fmv.w.x`, `fcvt.s.w`); and `Run()` paused at the end-of-program breakpoint while draining, so `kites-cli` never finished on a pipeline model.

9 Statistics and timing model

9.1 Statistics

After a successful commit the vector unit passes an `InstructionRecord` to the processor's `VectorProfiler`: the instruction, SEW, LMUL, `vl`, VLMAX, `vstart`, the number of body, active and masked-off elements, the bytes loaded and stored, and two 32-bit masks of the vector registers read and written (collected by the element accessors). The profiler counts instructions per class (configuration, load, store, integer, fixed point, floating point, reduction, mask, permutation, bit manipulation) and per mnemonic, the average `vl` and $\sum$ `vl`/$\sum$ VLMAX, active and masked-off elements, bytes moved, and SEW and LMUL use.

9.2 Timing model

The model estimates how long the vector part of a program takes on a decoupled vector unit. The unit issues vector instructions in program order, at most one per cycle, and not before the cycle in which the processor reached the instruction. Each unit class (integer ALU, multiply, divide, FP add, FP multiply, FP divide, load, store, reduction, permute, mask) has one pipelined unit with L lanes. With start-up latency λ for the unit and n body elements, an instruction:

- occupies its unit for $b = \lceil n/L \rceil$ cycles. Strided and indexed accesses move one element per cycle, mask logical operations process $64L$ bits per cycle, and reductions add $\log_2 L$ cycles to combine the lanes;
- starts at $s = \max(\text{issue}, s_{\text{prev}} + 1, \text{unit free}, \text{operands ready})$, where an operand is ready at the producer's first result $s_p + \lambda_p$ with chaining, or at its last result $s_p + \lambda_p + b_p$ without;
- finishes at $e = s + \lambda + b$, and its unit is free again at $s + b$.

The estimate is $\max e$ minus the issue cycle of the first vector instruction. The default latencies

follow the VMIPS example in Hennessy and Patterson [4]: 6 cycles for add, 7 for multiply, 20 for divide and 12 for load and store. Convoys also follow the textbook: consecutive instructions form one convoy until one needs a unit already used in the convoy or, without chaining, reads a register written in it. The number of convoys is the chime count.

Table 5: Worked example, `vl`=8: `vsetivli` issues at cycle 0, `vle32.v v8` at 1, `vadd.vv v16, v8, v8` at 2.

	4 lanes, chaining	4 lanes, no chaining	1 lane, chaining	1 lane, no chaining
load: start, first, last	1, 13, 15	1, 13, 15	1, 13, 21	1, 13, 21
add: start	13	15	13	21
estimate (cycles)	21	23	27	35
convoys	1	2	1	2

The model is an estimate for comparing code and configurations, not a cycle-accurate simulation. It does not model vector-to-scalar register dependencies, memory banks, or the scalar instructions' own timing.

9.3 Undo and reset

Each step record keeps the profiler state before and after a vector instruction. Undo restores the state from before, redo the state from after, and reset clears it, so the statistics always match the visible program state.

10 User interfaces

Register panel
The *Vector* section lists `v0`–`v31` in hex; the *Vector CSRs* section shows `vl`, the decoded `vtype`, `vstart`, `vxrm`, `vxsat`, `vcsr` and `vlenb`.

Vector tab
A table of the 32 registers split into elements at the current SEW or a chosen one, shown as hex, unsigned, signed or floating point (including half). Tail elements are dimmed, masked-off elements can be shaded, LMUL groups are banded, and elements changed since the last update are highlighted. The vector CSRs are shown above the table and the statistics and timing report below it. Updates are coalesced so a fast run does not flood the GUI thread.

Settings
Settings → *Vector Extension* edits the options of Table 3. They are saved with `QSettings` and applied at start-up before the processors are created.

Circuit view
Every processor design has a *Vector Unit* and a *Vector Regs* block with five wires: scalar operands and results to and from the register file, vector register reads and writes, and memory. The pipelines have a sixth wire from MEM/WB. The wires light up while a vector instruction executes (in WB on the pipelines). `tools/vector/add_vector_unit_to_circuits.py` adds the blocks without reformatting the design files.

CLI
The `kites-cli` options `--vlen`, `--vector-agnostic` and `--vector-ext` set the configuration, and a batch run also writes `vector_stats.json`. The REPL adds `vregs`/`vreg` (elements in any SEW and format), `vcsr`, `vconfig` and `vstats`.

Custom pseudo-instructions

An expansion line may use vector instructions. Each operand is checked against the instruction's operand list: vector, integer and FP registers, a `(base)` memory operand, immediate ranges, vtype fields and `v0.t`. Arguments are substituted inside `(…)` too.

Editor

The editor highlights vector mnemonics and registers and offers them in autocompletion; both read `resources/instructions/instructions.json`. The profiler counts vector instructions as `V-Type`.

11 Design decisions

The decisions D1–D14 of the scope document hold, with these refinements:

Table 6: Decisions added or revised during the implementation.

ID	Decision	Rationale
D12′	Supersedes D12: the pipelines execute vector and CSR instructions in WB, with a stall for dependent scalar instructions (Section 8).	One vector unit for every processor; precise exceptions and program-order memory access follow from the existing step order.
D14′	Supersedes D14: every vector mnemonic assembles; legality depends on the configuration at execution time.	Configuration (profile, extensions) can change without re-assembling; matches how hardware treats unsupported encodings.
D15	Zve* profiles, Zvfh, Zvbb/Zvkb and Zvbc are options; the default configuration is plain V with VLEN=128.	The defaults keep the behaviour of the scope; each option is validated against the specification's combination rules.
D16	Half precision is computed through double with a software rounding step.	Portable (no compiler half type needed), correctly rounded (Section 6.5), and RISC-V tininess detection.
D17	`vfredusum`/`vfwredusum` with no active element canonicalise a NaN scalar.	An allowed choice that matches Spike, the reference model.
D18	The timing model is analytic and separate from the processors' cycle counts.	Shows the effect of lanes and chaining without changing the existing cycle semantics of the single-cycle and pipeline models.
D19	Statistics are part of the undoable state.	Undo, redo and reset keep statistics consistent with the program state.

12 Known limitations

- `frm`=RMM rounds ties to even in floating-point arithmetic, because the host has no such mode. Conversions to integer honour RMM.
- For SEW 32 and 64, underflow uses the host's tininess rule. Half precision detects tininess after rounding.
- Scalar single and half values are kept zero-extended in the `f` registers, as in Kites' F extension (F18), not NaN-boxed. `.vf` operands use the low SEW bits.
- Kites has no traps: an illegal instruction or access fault stops the program, and `vstart` is non-zero only after an explicit CSR write.

- On the pipeline models without hazard detection, a scalar instruction that reads a vector or CSR result must be two instructions after it.
- The timing model is not cycle-accurate (Section 9).

13 Source files

Table 7: New and changed files.

File	Role
`src/processor/vector/vector_isa_table.inc`	generated encodings
`src/processor/vector/vector_isa.*`	ISA table, decode, operands, disassembly
`src/processor/vector/vtype.*`	vtype decode/encode, VLMAX, legality
`src/processor/vector/vector_config.*`	VLEN, policy, profile, extensions
`src/processor/vector/vector_unit.*`	`execute`, `ExecState`, legality helpers, commit
`src/processor/vector/vector_exec.h`	shared declarations of the families
`src/processor/vector/vector_exec_integer.cpp`	integer, fixed point, mask, reduction, permutation, Zvbb/Zvbc
`src/processor/vector/vector_exec_memory.cpp`	loads and stores
`src/processor/vector/vector_exec_float.cpp`	floating point, half precision
`src/processor/vector/vector_profiler.*`	statistics and timing model
`src/processor/vector/vector_format.*`	element formatting for the CLI and GUI
`src/assembler/parse_formats/v_formats.cpp`	vector parser, pseudo-instructions, CSR pseudos
`src/assembler/lexer.cpp`, `parser.*`, `code_generator.*`	vector tokens, dispatch, pre-encoded words
`src/processor/registers.*`	vector registers and CSRs
`src/processor/rvss/rvss_processor.*`	vector execution, undo/redo, CSR read-only check
`src/processor/rv5s/*`	WB execution, stall, CSRs, scalar fixes
`src/processor/processor_base.*`	step record, profiler, circuit wires
`src/ui/vector_tab/*`, `src/ui/dialogs/settings_dialog.*`	vector viewer, settings page
`src/ui/register_table/*`	Vector and Vector CSRs sections
`src/custom_pseudo_manager/*`	vector expansions
`cli/main.cpp`, `cli/repl.cpp`	flags and REPL commands
`resources/circuit_designs/*.json`	vector unit blocks and wires
`tools/vector/*`	table and golden generators, Spike harness, circuit script
`test/vector/*`, `test/rv5s_regression_test.cpp`	tests

References

[1] RISC-V International. *The RISC-V Instruction Set Manual, Volume I: Unprivileged Architecture*, "V" Standard Extension for Vector Operations, version 1.0, and the Zvfh, Zvbb, Zvkb and Zvbc extensions. https://docs.riscv.org/reference/isa/v20260120/unpriv/v-st-ext.html

[2] RISC-V International. *riscv-opcodes*. https://github.com/riscv/riscv-opcodes

[3] RISC-V Software. *Spike RISC-V ISA Simulator*. https://github.com/riscv-software-src/riscv-isa-sim

[4] J. L. Hennessy and D. A. Patterson. *Computer Architecture: A Quantitative Approach*, 6th ed., Chapter 4. Morgan Kaufmann, 2017.

[5] S. A. Figueroa. "When is double rounding innocuous?" *ACM SIGNUM Newsletter* 30(3), 1995.